

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

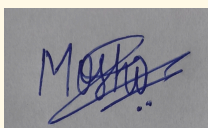
Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA0305	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Moshika M	Reg. No.:	192424064
Section / Batch:	B.Tech (AI-DS)	Date:	17.08.2026

Code of Conduct

I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

ANSWER:

A language processing system needs an effective way to represent the syntactic structure of sentences. Two common approaches are Context-Free Grammar (CFG) trees and dependency parsing. Although both represent sentence structure using trees, they focus on different aspects of syntax.

Context-Free Grammar (CFG) Trees

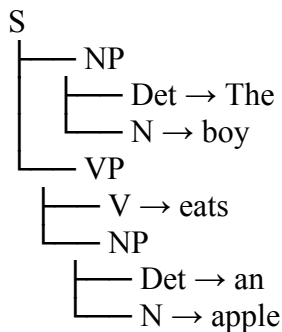
A CFG represents a sentence by dividing it into constituents or phrases. It describes how smaller grammatical units combine to form larger units.

For example:

Sentence:

The boy eats an apple.

A CFG may represent it as:



The CFG representation clearly shows that *The boy* forms a noun phrase and *eats an apple* forms a verb phrase.

CFG is especially useful for:

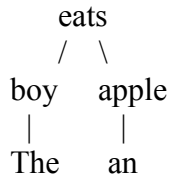
- Identifying phrases and constituents
- Grammar checking
- Syntactic analysis
- Sentence generation
- Representing hierarchical sentence structure

However, CFG does not directly represent the relationship between every pair of individual words. To determine that *boy* is the subject of *eats*, the system must interpret the structure of the tree.

Dependency Parsing

Dependency parsing focuses directly on the relationships between words. Each word is connected to another word that acts as its syntactic head.

For the same sentence:



The relationships can be represented as:

- **eats** → **boy** = subject
- **eats** → **apple** = object
- **boy** → **The** = determiner
- **apple** → **an** = determiner

Dependency parsing therefore directly captures grammatical relationships such as:

- Subject
- Object
- Modifier
- Determiner
- Complement
- Auxiliary
- Prepositional attachment

Analysis

CFG is particularly useful when the goal is to understand how words form larger phrases. For example, it can distinguish noun phrases, verb phrases, prepositional phrases, and clauses.

Dependency parsing is more concerned with who depends on whom. This makes it especially useful for applications such as information extraction, question answering, machine translation, and semantic analysis.

For example:

“The student submitted the assignment.”

Dependency parsing can directly identify:

submitted → **student** = **subject**
submitted → **assignment** = **object**

This is valuable because many NLP applications need relationships between individual words rather than only phrase boundaries.

Comparison

Feature	CFG Trees	Dependency Parsing
Basic unit	Phrase/constituent	Word
Main representation	Phrase hierarchy	Head-dependent relationships
Subject identification	Indirect	Direct
Object identification	Indirect	Direct
Phrase structure	Excellent	Less explicit
Word relationships	Less direct	Very explicit
Free word order	Less convenient	More flexible
Grammar checking	Useful	Very useful
Information extraction	Moderate	Strong
Semantic relation support	Limited	Better

Conclusion

Dependency parsing is more effective for capturing relationships between words, because it explicitly represents head-dependent relationships. However, CFG trees remain valuable when phrase-level and hierarchical structure is important. In a modern NLP system, the two approaches can also complement each other.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

ANSWER:

Parsing is the process of determining the syntactic structure of an input sentence according to a grammar. Top-down parsing and Earley parsing are two approaches with different strategies for constructing this structure.

Top-Down Parsing

Top-down parsing begins with the start symbol, usually **S**, and attempts to generate the input sentence by applying grammar rules.

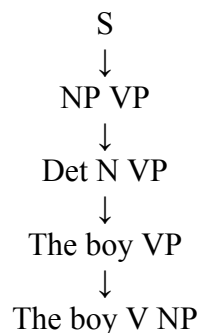
For example:

$S \rightarrow NP VP$
 $VP \rightarrow V NP$
 $NP \rightarrow Det N$

For:

“The boy eats rice.”

The parser starts with:



It continues until the generated structure matches the input.

Advantages of Top-Down Parsing

- Simple to understand
- Easy to implement
- Works well for simple grammars
- Starts with the expected sentence structure
- Can be efficient when grammar choices are limited

Limitations

Top-down parsing can become inefficient when:

- The grammar contains many alternatives.
- The sentence is ambiguous.
- The input is incomplete.
- The parser predicts structures that do not match the input.
- There is left recursion in the grammar.

For example, if the user enters:

“The student who...”

the sentence is incomplete. A basic top-down parser may have difficulty determining how the incomplete structure should be completed.

Earley Parsing

Earley parsing is a dynamic programming algorithm for parsing context-free grammars. It processes the input from left to right while maintaining partial grammatical structures.

It uses three major operations:

1. Prediction

The parser predicts which grammar rules could occur next.

2. Scanning

It checks whether the predicted terminal symbol matches the next input word.

3. Completion

When a grammar rule has been completely recognized, the parser updates previously predicted structures.

For example:

Input:

The student reads

Instead of completely discarding partial structures, Earley parsing can maintain incomplete states such as:

$S \rightarrow NP \bullet VP$

$VP \rightarrow V \bullet NP$

The dot indicates how much of a rule has already been recognized.

Comparison

Feature	Top-Down Parsing	Earley Parsing
Parsing direction	Start symbol toward input	Input processed left-to-right
Partial input	Less suitable	Highly suitable
Ambiguous input	Can require extensive backtracking	Maintains multiple possibilities
Context-free grammars	Works with suitable CFGs	Handles general CFGs
Left recursion	Problematic in basic versions	Supported
Dynamic programming	Usually no	Yes
Memory usage	Generally lower	Higher
Implementation	Simpler	More complex
Flexibility	Lower	Higher

Analysis for Real-Time Applications

Consider a voice assistant or chatbot where users may enter:

- “Book a flight to...”
- “Show me the transactions from...”
- “I want to cancel my...”
- “Can you tell me whether...”

The input may be incomplete because the user is still typing or speaking.

Earley parsing is better suited to these situations because it can maintain partial parses and multiple possible interpretations.

For ambiguous input, it can also preserve different possible structures rather than immediately committing to one interpretation.

Conclusion

For simple and predictable grammars, top-down parsing may be faster and easier to implement. However, for dynamic applications involving incomplete, ambiguous, or complex input, Earley parsing is more suitable because it uses dynamic programming and maintains partial parsing states.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

ANSWER:

Natural language is inherently ambiguous. A single sentence can have multiple syntactic interpretations.

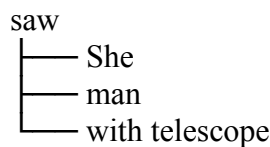
Consider:

“She saw the man with a telescope.”

There are at least two possible interpretations:

Interpretation 1

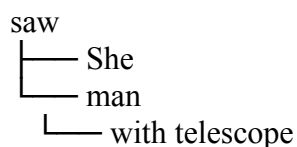
She used a telescope to see the man.



Here, *with a telescope* modifies the action *saw*.

Interpretation 2

The man had a telescope.



Here, *with a telescope* modifies *the man*.

The three approaches—CFG, PCFG, and neural parsing—handle this ambiguity differently.

Context-Free Grammar

A CFG defines syntactic rules such as:

$S \rightarrow NP VP$
 $VP \rightarrow V NP$
 $VP \rightarrow VP PP$
 $NP \rightarrow NP PP$

The grammar can potentially generate both interpretations.

The major problem is that CFG itself does not assign probabilities to the alternative structures.

Therefore, if two parse trees are grammatically valid, CFG cannot determine which interpretation is more likely without additional information.

Advantages

- Simple
- Linguistically interpretable
- Requires relatively little training data
- Useful for controlled language

Limitations

- Cannot naturally rank ambiguous parses
- Limited contextual understanding
- Requires manually designed grammar rules

Probabilistic Context-Free Grammar

A PCFG extends CFG by assigning probabilities to grammar rules.

For example:

VP $\rightarrow$ VP PP 0.30
NP $\rightarrow$ NP PP 0.70

The parser calculates the probability of different possible parse trees.

If the training corpus indicates that a prepositional phrase is more commonly attached to a noun, the corresponding parse can receive a higher probability.

Advantages

- Handles ambiguity better than ordinary CFG
- Can rank alternative parse trees
- Relatively interpretable
- Uses statistical information from data

Limitations

- Relies heavily on grammar-rule probabilities
- Basic PCFGs have limited contextual information
- May not capture long-range dependencies
- Performance depends on the quality of the training corpus

Neural Parsing

Neural parsers use machine-learning architectures to learn syntactic patterns from large datasets.

Modern neural parsers can consider:

- Surrounding words
- Sentence context
- Word representations
- Long-distance dependencies
- Syntactic patterns
- Contextual meaning

For example, the meaning of:

“with a telescope”

can be influenced by the surrounding context.

Compare:

“She saw the man with a telescope.”

with:

“She saw the man with a beard.”

A neural model can learn that *with a beard* is more naturally associated with the person, while *with a telescope* is more naturally associated with the action in many contexts.

Comparison

Feature	CFG	PCFG	Neural Parsing
Basic grammar rules	Yes	Yes	Learned
Handles ambiguity	Poorly	Better	Very well
Probabilities	No	Yes	Yes/scores
Context awareness	Low	Limited	High
Long-range relationships	Limited	Limited	Strong
Training data	Low	Moderate	Usually high
Interpretability	High	High/medium	Lower
Real-world accuracy	Lower	Moderate	Generally higher

Conclusion

For modern real-world NLP applications, neural parsing is generally the most effective because it can use rich contextual information and learn complex patterns from large datasets.

PCFG provides a useful intermediate solution when probabilistic ranking is required but computational simplicity and interpretability are important. CFG is best suited to controlled or rule-based grammatical systems.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

ANSWER:

Grammar systems must ensure that sentences follow grammatical rules. Two important mechanisms are feature structures and subcategorization frames.

They address different aspects of grammatical correctness.

Feature Structures

A feature structure represents grammatical information using a collection of attributes and values.

For example:

Subject:

[Person = 3
Number = singular]

Verb:

[Person = 3
Number = singular]

The grammar can require the subject and verb to have compatible features.

Example

Correct:

She walks to college.

She → Number = singular
walks → Number = singular

Incorrect:

She walk to college.

She → Number = singular
walk → Number = plural

The feature structure system can detect this mismatch.

Feature structures can represent:

- Number
- Person
- Gender
- Tense

- Case
- Mood
- Voice
- Agreement
- Definiteness

They can therefore support a wide range of grammatical constraints.

Subcategorization Frames

A subcategorization frame specifies the arguments or complements that a particular verb requires.

For example:

“Eat”

eat → NP

Example:

She eats rice.

“Give”

give → NP NP

Example:

She gave me a book.

“Depend”

depend → PP(on)

Example:

Success depends on effort.

The frame tells the parser what kinds of complements are expected after a verb.

Importance

If the parser encounters:

She depends success.

the subcategorization information can identify a problem because depend normally requires a prepositional complement such as *on success*.

Comparison

Feature	Feature Structures	Subcategorization Frames
Main purpose	Represent grammatical features	Represent verb argument requirements
Subject–verb agreement	Excellent	Limited
Person	Supported	Usually not primary
Number	Supported	Usually not primary
Tense	Supported	Not primary
Verb complements	Can represent	Excellent
Agreement checking	Strong	Weak
Lexical information	Moderate	Strong
Grammar constraints	Broad	Verb-specific

Analysis

Feature structures provide a more general mechanism for enforcing grammatical constraints because they can represent and compare multiple linguistic properties.

Subcategorization frames are narrower but extremely important for verb argument structure.

For example: The students gives the teacher a book.

The feature structure can detect:

students → plural

gives → singular

Therefore, agreement is incorrect. At the same time, a subcategorization frame for *give* can ensure that the verb receives the appropriate arguments:

give → NP + NP + NP

Thus, the two approaches solve different problems.

Conclusion

If the primary requirement is subject–verb agreement and general grammatical correctness, feature structures are more effective. If the requirement is specifically to ensure that verbs receive the correct complements and arguments, subcategorization frames are more effective.

A sophisticated grammar system should ideally use both together.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

ANSWER:

Dependency parsing is widely used in NLP systems to identify relationships between words. Two important approaches are transition-based parsing and graph-based parsing.

Transition-Based Parsing

Transition-based parsing builds the dependency tree incrementally, using a sequence of actions.

A parser generally maintains:

- A stack
- An input buffer
- A partially constructed dependency tree

Common operations include:

SHIFT

Moves a word from the input buffer to the stack.

REDUCE

Removes a word from the stack after processing it.

LEFT-ARC

Creates a dependency where the current word is related to another word.

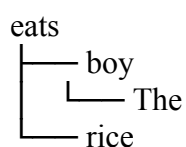
RIGHT-ARC

Creates the opposite dependency relationship.

For example:

“The boy eats rice.”

The parser processes the words step by step and gradually constructs:



Advantages

- Very fast

- Low computational cost
- Suitable for real-time processing
- Works well with large datasets
- Can use machine-learning classifiers to predict transitions

Limitation

Its major limitation is local decision-making.

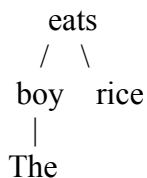
Once the parser makes an incorrect transition, that mistake can affect subsequent decisions.

Graph-Based Parsing

Graph-based parsing treats the dependency tree as a global optimization problem.

Instead of deciding only what action should happen next, the parser considers possible relationships between words and attempts to find the best overall dependency tree.

For example:



The parser assigns scores to possible dependency edges and searches for the tree with the highest overall score.

Advantages

- Makes global decisions
- Can consider relationships across the whole sentence
- Less affected by individual local decisions
- Often strong for complex dependency structures
- Handles long-distance relationships effectively

Limitations

- Computationally more expensive
- Generally slower than transition-based methods
- Requires more memory
- Global optimization can become costly for large datasets

Analysis for Large-Scale NLP

Suppose an NLP system must process millions of sentences from:

- Search engines
- Social media
- Customer support
- News articles

- Large document collections

Processing speed becomes extremely important.

A transition-based parser can process sentences quickly because it makes a sequence of relatively inexpensive decisions.

For example:

Input → SHIFT → ARC → SHIFT → REDUCE → ARC → Output

This makes it suitable for high-throughput systems.

Graph-based parsing may provide stronger global analysis, but the additional computational cost can become significant when processing millions of sentences.

Comparison

Feature	Transition-Based	Graph-Based
Decision type	Local	Global
Parsing method	Sequence of transitions	Optimization over dependency trees
Speed	Very high	Lower
Memory requirement	Lower	Higher
Scalability	Excellent	Moderate
Error propagation	More likely	Less likely
Long-distance dependencies	Can be challenging	Generally better
Real-time applications	Excellent	Less suitable
Large datasets	Highly suitable	More computationally demanding
Accuracy	Very good	Often strong

Conclusion

For large-scale applications where speed, memory efficiency, and scalability are major requirements, transition-based dependency parsing is generally more suitable.

However, if the application prioritizes global structural accuracy over processing speed—for example, detailed linguistic analysis—graph-based parsing may be preferred.

A practical modern system can also combine the strengths of both approaches by using neural models to improve transition decisions or by applying global reranking after fast initial parsing.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____